



Jose Rizal Memorial State University-Main Campus
The Premier University in Zamboanga Del Norte
College of Computing Studies



Platform-Based Development

Multi-Dimensional Token ID (MDTID)

An Improvised Token Identification Framework for Natural Language Processing

Submitted by:

KHENT BRYAN JACALAN
LEO ALFIER ANTIPUESTO
RALDIN CASIDAR
LAWRENCE SALDO

Submitted to:

MR. GEHAN L. RESALUTE

May 18, 2026

1. Abstract

Multi-Dimensional Token ID (MDTID) is a proposed improvement over current flat integer-based token identification systems used in Natural Language Processing. Rather than assigning a single arbitrary integer to each token, MDTID encodes multiple linguistic properties directly into the token ID itself — enabling models to access richer information from the very first layer of processing.

2. Introduction

Tokenization is the foundational step in any Natural Language Processing (NLP) pipeline. It is the process of converting raw text into discrete units called tokens, which are then assigned numerical identifiers for machine processing. These numerical identifiers are known as Token IDs.

Current Token ID implementations, such as those used by Byte-Pair Encoding (BPE), WordPiece, and SentencePiece, assign each token a single flat integer from a fixed vocabulary. While this approach is simple and computationally efficient, it carries significant limitations:

- Token IDs are arbitrary — the number 4821 assigned to "dog" carries no semantic meaning whatsoever.
- Token IDs are context-blind — the word "bank" always receives the same ID regardless of whether it refers to a financial institution or a riverbank.
- Token IDs are static — vocabularies are frozen at training time and cannot adapt to new words or evolving language.
- Token IDs carry no relational structure — there is no mathematical relationship between the IDs of related words like "dog" and "puppy".

The Multi-Dimensional Token ID (MDTID) framework proposes to address all of these limitations by encoding multiple linguistic dimensions directly within each token's identifier.

3. Background: Current Token ID Implementation

3.1 How Token IDs Work Today

In current NLP systems, tokenization follows a two-stage process:

- Stage 1 — Tokenization: Raw text is split into tokens (words, subwords, or characters) using algorithms such as BPE or WordPiece.
- Stage 2 — Encoding: Each token is mapped to a unique integer ID using a fixed vocabulary lookup table.

For example, given the sentence "machine learning is powerful":

Token	Assigned ID	Meaning of ID
machine	4821	Arbitrary integer — carries no information
learning	6103	Arbitrary integer — carries no information
is	318	Arbitrary integer — carries no information
powerful	9442	Arbitrary integer — carries no information

The resulting sequence [4821, 6103, 318, 9442] is then passed to the model, which must learn all semantic, grammatical, and contextual relationships entirely from training data — receiving no prior information from the IDs themselves.

3.2 Key Problems with Flat Token IDs

Problem	Description	Impact
Arbitrariness	IDs carry no linguistic meaning	Model learns from scratch every time
Context Blindness	Same word always maps to same ID	Polysemy is unresolved at token level
Static Vocabulary	New words fragment into subwords	Poor handling of emerging language
No Relational Structure	Related words have unrelated IDs	Analogical reasoning must be fully learned
Domain Insensitivity	No distinction between technical domains	Extra burden on model for domain tasks

4. Proposed Framework: Multi-Dimensional Token ID (MDTID)

4.1 Core Concept

The central idea of MDTID is simple: instead of assigning a single meaningless integer to each token, the token ID itself is structured to encode multiple linguistic properties simultaneously.

Core Principle: Every bit of information that can be known about a token at tokenization time should be embedded into the token ID — so the model receives a richer signal from the very beginning, before any computation begins.

4.2 ID Structure and Dimensions

An MDTID is a structured 32-bit identifier composed of the following dimensions:

Dimension	Bits Allocated	Purpose	Example Values
Concept Base ID	16 bits	Core lexical identity of the token	11042 = root concept of 'diagnose'
Part of Speech (POS)	4 bits	Grammatical category	0001=noun, 0010=verb, 0011=adjective
Tense / Morphological Form	4 bits	Verb tense or word form	0001=base, 0010=past, 0011=present
Domain	4 bits	Subject matter domain	0000=general, 0001=medical, 0010=legal
Sentiment Polarity	2 bits	Emotional/tonal orientation	00=neutral, 01=positive, 10=negative
Formality Level	1 bit	Register of language use	0=informal, 1=formal use
Language Tag	1 bit	Language of origin flag	0=English, 1=cross-lingual anchor

4.3 Example: Encoding the Word "diagnosed"

Consider the token "diagnosed" appearing in a formal medical report:

Dimension	Value	Encoded Bits
Concept Base ID	11042 (diagnose root)	0010101100100010
Part of Speech	Verb	0010
Tense / Form	Past tense	0010
Domain	Medical	0001
Sentiment Polarity	Negative lean	10

Formality Level	Formal	1
Language Tag	English	0

Final MDTID: 0010101100100010 | 0010 | 0010 | 0001 | 10 | 1 | 0

Compare this to the current approach where "diagnosed" simply maps to an arbitrary number like 9371 — providing the model with zero prior information about the token's properties.

5. Dimension Specifications

5.1 Part of Speech (POS)

The POS dimension encodes the grammatical role of the token. This eliminates the need for the model to deduce POS from context alone, significantly reducing the learning burden for tasks such as parsing, named entity recognition, and machine translation.

POS Code	Binary	Meaning
0	0000	Unknown / Unassigned
1	0001	Noun
2	0010	Verb
3	0011	Adjective
4	0100	Adverb
5	0101	Pronoun
6	0110	Preposition
7	0111	Conjunction
8	1000	Determiner
9	1001	Punctuation / Symbol

5.2 Tense / Morphological Form

This dimension captures the inflectional form of a token, providing verb tense, number (singular/plural), and comparative degree information directly at the tokenization stage.

Code	Binary	Meaning
0	0000	Base / Infinitive form
1	0001	Present tense

2	0010	Past tense
3	0011	Future / Modal
4	0100	Plural form
5	0101	Comparative (e.g., faster)
6	0110	Superlative (e.g., fastest)
7	0111	Gerund / Present participle

5.3 Domain

The domain dimension encodes the subject matter context of the token. This is especially valuable for specialized NLP applications where general-purpose models struggle with domain-specific vocabulary.

Code	Binary	Domain
0	0000	General / Common language
1	0001	Medical / Healthcare
2	0010	Legal
3	0011	Technology / Computer Science
4	0100	Finance / Economics
5	0101	Science / Academic
6	0110	News / Journalism
7	0111	Informal / Social media

5.4 Sentiment Polarity

The sentiment dimension provides a coarse emotional orientation of the token, allowing sentiment-sensitive downstream tasks to access this information without requiring a full sentiment analysis pass over the sequence.

Code	Binary	Meaning
0	00	Neutral — no sentiment signal
1	01	Positive polarity
2	10	Negative polarity
3	11	Mixed or contextual

5.5 Formality Level

A single bit indicating whether the token belongs to formal or informal register. This helps models tasked with style transfer, professional document generation, or social media analysis to correctly handle register differences from the tokenization stage onward.

5.6 Language Tag

A single bit flag that, when set, marks the token as a cross-lingual concept anchor — meaning its Concept Base ID is shared with equivalent tokens in other languages. This is the first step toward a true Universal Concept Vocabulary across languages.

6. Advantages Over Current Token IDs

Advantage	Current System	MDTID
Information per token	Single integer (0 bits of linguistic info)	32 structured bits of linguistic info
Polysemy handling	Same ID for all meanings of a word	Domain + context bits differentiate meanings
Model training efficiency	Model learns all properties from scratch	Properties pre-encoded; model focuses on reasoning
Domain adaptation	Requires full retraining or fine-tuning	Domain bit allows zero-shot domain awareness
Sentiment tasks	Sentiment learned only from large corpora	Sentiment signal present from token 1
Cross-lingual transfer	Separate vocabularies per language	Shared concept IDs enable cross-lingual mapping
Model size requirements	Large models needed to learn basic features	Smaller models possible with richer input signal

7. Limitations and Challenges

7.1 Tokenizer Complexity

Building an MDTID tokenizer requires significantly more engineering than current approaches. Each token must be annotated with POS, domain, sentiment, and other dimensions at vocabulary construction time, requiring large-scale linguistic annotation pipelines.

7.2 Annotation Errors Propagate

Because linguistic properties are baked into the token ID, errors made during vocabulary construction will propagate through every inference call. A word incorrectly tagged as "medical domain" will always carry that incorrect tag.

7.3 Ambiguity at Tokenization Time

Some dimensions — particularly domain and POS — are genuinely context-dependent. Assigning them at tokenization time introduces a layer of ambiguity that context-dependent systems (like attention mechanisms) currently resolve more flexibly.

7.4 Vocabulary Management

The multi-dimensional structure increases vocabulary management complexity. Adding a new word requires not just a new Concept Base ID but also the correct assignment of all dimensional values — a process that requires expert annotation.

Challenge	Severity	Potential Mitigation
Tokenizer construction complexity	High	Automated NLP annotation pipelines
Annotation errors propagate	Medium	Confidence scores + fallback to neutral values
Context-dependent dimension ambiguity	Medium	Default to neutral; let model override via attention
Vocabulary management overhead	Medium	Dynamic vocabulary expansion with automated tagging
Backward compatibility	Low	MDTID is additive; base ID alone = current behavior

8. Comparison with Related Approaches

Approach	Solves	Does Not Solve	MDTID Advantage
BPE (GPT-4)	Compression efficiency	Meaning, context, relationships	MDTID encodes all three
WordPiece (BERT)	Subword coverage	Polysemy, domain awareness	MDTID adds domain + POS bits
SentencePiece (T5)	Multilingual tokenization	Semantic relationships across languages	MDTID cross-lingual tag achieves this
ByT5 (byte-level)	No OOV tokens	Still arbitrary byte IDs	MDTID gives meaning to every ID

One-Hot Encoding	Distinct token representation	Wastes space, no linguistic info	MDTID is compact and information-rich
------------------	-------------------------------	----------------------------------	---------------------------------------

9. Practical Use Cases

9.1 Medical NLP

With the domain bit set to "medical" at tokenization time, a model processing clinical notes immediately knows which tokens belong to the medical lexicon — without needing to learn this distinction entirely from training data. This is especially valuable in low-resource medical NLP settings where training data is scarce.

9.2 Sentiment Analysis

The sentiment dimension provides a coarse prior for every token in the input sequence. Sentiment analysis models can treat this as a strong feature, reducing their dependence on large labeled datasets and improving performance in zero-shot or few-shot settings.

9.3 Machine Translation

By sharing Concept Base IDs across languages (via the language tag bit), MDTID enables a form of lexical transfer at the tokenization layer. A model trained on English data can leverage shared concept IDs when processing Spanish or French tokens it has never seen before.

9.4 Code Generation

When the domain bit is set to "technology", tokens drawn from programming vocabulary are immediately flagged as technical. This helps code generation models avoid treating reserved keywords like "class", "return", or "null" as general English words.

10. Code Examples

This section demonstrates the MDTID framework through both pseudocode (conceptual) and Python (implementation) examples. The pseudocode focuses on readability and logic flow, while the Python examples show how the framework can be implemented in practice.

10.1 Pseudocode Examples

Example 1 — Defining the MDTID Structure

This pseudocode defines the core data structure of a Multi-Dimensional Token ID and how its fields map to bit positions in a 32-bit integer.

PSEUDOCODE — MDTID Structure Definition	
STRUCTURE MDTID:	
concept_base_id	: 16 bits // Core lexical identity
pos	: 4 bits // Part of Speech
tense	: 4 bits // Tense / Morphological Form
domain	: 4 bits // Subject domain
sentiment	: 2 bits // Emotional polarity
formality	: 1 bit // Formal vs informal
language_tag	: 1 bit // Cross-lingual flag
TOTAL	: 32 bits
CONSTANTS:	
POS	= { UNKNOWN:0, NOUN:1, VERB:2, ADJ:3, ADV:4, ... }
TENSE	= { BASE:0, PRESENT:1, PAST:2, FUTURE:3, PLURAL:4, ... }
DOMAIN	= { GENERAL:0, MEDICAL:1, LEGAL:2, TECH:3, FINANCE:4, ... }
SENTIM	= { NEUTRAL:0, POSITIVE:1, NEGATIVE:2, MIXED:3 }

Example 2 — Encoding a Token into MDTID

This pseudocode shows the step-by-step encoding process that converts a raw token and its properties into a single 32-bit MDTID integer.

PSEUDOCODE — MDTID Encoding Algorithm	
FUNCTION encode_mdtid(token, pos, tense, domain, sentiment, formality, language_tag):	
// Step 1: Look up the concept base ID from vocabulary	
base_id = VOCABULARY.lookup(token)	
IF base_id NOT FOUND:	
base_id = VOCABULARY.add_new(token) // dynamic expansion	
// Step 2: Pack all dimensions into 32-bit integer	
mdtid = 0	
mdtid = mdtid OR (base_id << 16) // bits 31..16	
mdtid = mdtid OR (pos << 12) // bits 15..12	
mdtid = mdtid OR (tense << 8) // bits 11..8	
mdtid = mdtid OR (domain << 4) // bits 7..4	
mdtid = mdtid OR (sentiment << 2) // bits 3..2	
mdtid = mdtid OR (formality << 1) // bit 1	
mdtid = mdtid OR (language_tag) // bit 0	
RETURN mdtid	
// Example usage:	
id = encode_mdtid(	
token = 'diagnosed',	

pos	=	VERB,
tense	=	PAST,
domain	=	MEDICAL,
sentiment	=	NEGATIVE,
formality	=	1,
language_tag	=	0
)		
// Result: 32-bit MDTID encoding all properties of 'diagnosed'		

Example 3 — Decoding an MDTID Back to Properties

The reverse operation: given a packed 32-bit MDTID, extract all individual dimension values using bitmasking.

PSEUDOCODE — MDTID Decoding Algorithm	
FUNCTION decode_mdtid(mdtid):	
// Extract each dimension using bitmasks and bit shifts	
concept_base	= (mdtid >> 16) AND 0xFFFF // top 16 bits
pos	= (mdtid >> 12) AND 0x000F // bits 15..12
tense	= (mdtid >> 8) AND 0x000F // bits 11..8
domain	= (mdtid >> 4) AND 0x000F // bits 7..4
sentiment	= (mdtid >> 2) AND 0x0003 // bits 3..2
formality	= (mdtid >> 1) AND 0x0001 // bit 1
language_tag	= mdtid AND 0x0001 // bit 0
RETURN {	
token	: VOCABULARY.reverse_lookup(concept_base),
pos	: POS_NAMES[pos],
tense	: TENSE_NAMES[tense],
domain	: DOMAIN_NAMES[domain],
sentiment	: SENTIMENT_NAMES[sentiment],
formality	: IF formality == 1 THEN 'formal' ELSE 'informal',
language	: IF language_tag == 1 THEN 'cross-lingual' ELSE 'English'
}	

Example 4 — Tokenizing a Full Sentence

This pseudocode shows how MDTID encoding is applied across an entire input sentence, producing a sequence of structured token IDs.

PSEUDOCODE — Sentence Tokenization with MDTID
FUNCTION tokenize_sentence(sentence):
tokens = split_into_tokens(sentence) // basic tokenization
mdtid_sequence = []
FOR EACH token IN tokens:

// Annotate token properties (via lookup or tagger)
props = ANNOTATOR.tag(token)
// Encode as MDTID
mdtid = encode_mdtid(
token = token,
pos = props.pos,
tense = props.tense,
domain = props.domain,
sentiment = props.sentiment,
formality = props.formality,
language_tag = props.language_tag
)
mdtid_sequence.APPEND(mdtid)
RETURN mdtid_sequence
// Example:
// Input: 'The patient was diagnosed with hypertension'
// Output: [MDTID_the, MDTID_patient, MDTID_was,
// MDTID_diagnosed, MDTID_with, MDTID_hypertension]
// where each MDTID is a packed 32-bit integer with full annotations

10.2 Python Implementation Examples

Example 1 — MDTID Data Class and Constants

A Python dataclass representing the MDTID structure, along with constant dictionaries for each dimension.

PYTHON — MDTID Data Class and Constants
from dataclasses import dataclass
from typing import Dict, Optional
--- Dimension Constants ---
POS = {
'UNKNOWN': 0, 'NOUN': 1, 'VERB': 2, 'ADJ': 3,
'ADV': 4, 'PRON': 5, 'PREP': 6, 'CONJ': 7,
'DET': 8, 'PUNCT': 9
}
TENSE = {
'BASE': 0, 'PRESENT': 1, 'PAST': 2,
'FUTURE': 3, 'PLURAL': 4, 'COMPARATIVE': 5,
'SUPERLATIVE': 6, 'GERUND': 7
}

```

DOMAIN = {
    'GENERAL': 0, 'MEDICAL': 1, 'LEGAL': 2,
    'TECH': 3, 'FINANCE': 4, 'SCIENCE': 5,
    'NEWS': 6, 'INFORMAL': 7
}

SENTIMENT = {
    'NEUTRAL': 0, 'POSITIVE': 1,
    'NEGATIVE': 2, 'MIXED': 3
}

# --- MDTID Data Structure ---
@dataclass
class MDTIDToken:
    token: str
    concept_id: int # 16-bit base concept ID
    pos: int # 4 bits
    tense: int # 4 bits
    domain: int # 4 bits
    sentiment: int # 2 bits
    formality: int # 1 bit
    language_tag: int # 1 bit

```

Example 2 — Encoding and Decoding Functions

Core functions to pack an MDTIDToken into a single 32-bit integer, and to unpack it back into its components.

```

PYTHON — Encoding and Decoding

def encode(token: MDTIDToken) -> int:
    """Pack all MDTID dimensions into a single 32-bit integer."""
    mdtid = 0
    mdtid |= (token.concept_id & 0xFFFF) << 16 # bits 31..16
    mdtid |= (token.pos & 0x000F) << 12 # bits 15..12
    mdtid |= (token.tense & 0x000F) << 8 # bits 11..8
    mdtid |= (token.domain & 0x000F) << 4 # bits 7..4
    mdtid |= (token.sentiment & 0x0003) << 2 # bits 3..2
    mdtid |= (token.formality & 0x0001) << 1 # bit 1
    mdtid |= (token.language_tag & 0x0001) # bit 0
    return mdtid

def decode(mdtid: int, vocab_reverse: Dict[int, str]) -> dict:
    """Unpack a 32-bit MDTID into its individual dimension values."""
    POS_NAMES = {v: k for k, v in POS.items()}
    TNS_NAMES = {v: k for k, v in TENSE.items()}
    DOM_NAMES = {v: k for k, v in DOMAIN.items()}
    SENT_NAMES = {v: k for k, v in SENTIMENT.items()}

```

```

concept_id = (mdtid >> 16) & 0xFFFF
pos         = (mdtid >> 12) & 0x000F
tense       = (mdtid >> 8) & 0x000F
domain      = (mdtid >> 4) & 0x000F
sentiment   = (mdtid >> 2) & 0x0003
formality   = (mdtid >> 1) & 0x0001
language_tag = mdtid & 0x0001

return {
    'token'      : vocab_reverse.get(concept_id, '<UNK>'),
    'pos'        : POS_NAMES.get(pos, 'UNKNOWN'),
    'tense'      : TNS_NAMES.get(tense, 'BASE'),
    'domain'     : DOM_NAMES.get(domain, 'GENERAL'),
    'sentiment'  : SENT_NAMES.get(sentiment, 'NEUTRAL'),
    'formality'  : 'formal' if formality else 'informal',
    'language'   : 'cross-lingual' if language_tag else 'English'
}

```

Example 3 — MDTID Tokenizer Class

A complete tokenizer class that manages a vocabulary, encodes tokens to MDTID integers, and decodes them back.

PYTHON — MDTIDTokenizer Class
<pre> class MDTIDTokenizer: def __init__(self): self.vocab: Dict[str, int] = {} # token -> concept_id self.vocab_reverse: Dict[int, str] = {} self._next_id = 1 def _get_or_add(self, token: str) -> int: """Look up or dynamically assign a concept base ID.""" if token not in self.vocab: self.vocab[token] = self._next_id self.vocab_reverse[self._next_id] = token self._next_id += 1 return self.vocab[token] def encode_token(self, token: str, pos: str = 'NOUN', tense: str = 'BASE', domain: str = 'GENERAL', sentiment: str = 'NEUTRAL', formality: int = 0, language_tag: int = 0) -> int: """Encode a single token into a 32-bit MDTID integer.""" t = MDTIDToken(token = token, concept_id = self._get_or_add(token), pos = POS.get(pos.upper(), 0), tense = TENSE.get(tense.upper(), 0), domain = DOMAIN.get(domain.upper(), 0), sentiment = SENTIMENT.get(sentiment.upper(), 0), </pre>

```

        formality    = formality,
        language_tag = language_tag
    )
    return encode(t)

def decode_token(self, mdtid: int) -> dict:
    """Decode a 32-bit MDTID back into its dimension values."""
    return decode(mdtid, self.vocab_reverse)

def tokenize(self, words_with_props: list) -> list:
    """Tokenize a list of (token, props_dict) tuples."""
    return [
        self.encode_token(word, **props)
        for word, props in words_with_props
    ]

```

Example 4 — Full End-to-End Usage

A complete working example showing how to tokenize a medical sentence using MDTID, inspect the resulting encoded IDs, and decode them back into human-readable properties.

PYTHON — End-to-End Tokenization Example
Initialize tokenizer
tokenizer = MDTIDTokenizer()
Sentence: 'The patient was diagnosed with hypertension'
Each token is annotated with its linguistic properties
sentence = [
('the', {'pos':'DET', 'tense':'BASE', 'domain':'GENERAL',
'sentiment':'NEUTRAL', 'formality':1}),
('patient', {'pos':'NOUN', 'tense':'BASE', 'domain':'MEDICAL',
'sentiment':'NEUTRAL', 'formality':1}),
('was', {'pos':'VERB', 'tense':'PAST', 'domain':'GENERAL',
'sentiment':'NEUTRAL', 'formality':1}),
('diagnosed', {'pos':'VERB', 'tense':'PAST', 'domain':'MEDICAL',
'sentiment':'NEGATIVE', 'formality':1}),
('with', {'pos':'PREP', 'tense':'BASE', 'domain':'GENERAL',
'sentiment':'NEUTRAL', 'formality':1}),
('hypertension', {'pos':'NOUN', 'tense':'BASE', 'domain':'MEDICAL',
'sentiment':'NEGATIVE', 'formality':1}),
]
Encode to MDTID sequence
mdtid_sequence = tokenizer.tokenize(sentence)
print('=== MDTID Encoded Sequence ===')
for i, mdtid in enumerate(mdtid_sequence):
print(f'Token {i+1}: {mdtid} (0x{mdtid:08X})')


```
compare_flat_vs_mdtd(tokenizer, 'bank',  
    pos='NOUN', tense='BASE', domain='GENERAL',  
    sentiment='NEUTRAL', formality=0)  
# Note: 'bank' in FINANCE vs GENERAL gets DIFFERENT MDTIDs  
# but would get the SAME flat Token ID – this is the key advantage
```

11. Conclusion

The Multi-Dimensional Token ID (MDTID) framework represents a fundamental rethinking of what a token identifier can and should be. Rather than treating Token IDs as arbitrary lookup keys, MDTID treats them as structured carriers of linguistic knowledge.

By encoding Part of Speech, Tense, Domain, Sentiment, Formality, and Language information directly into the 32-bit token identifier, MDTID provides every downstream model with a richer, more informative input signal — enabling faster training, smaller model sizes, better domain adaptation, and more accurate performance on specialized NLP tasks.

The most significant insight of MDTID is this: every computation cycle a model spends inferring basic properties of a token is a cycle that could have been saved if those properties were already encoded in the token's identifier. MDTID makes this optimization possible.

While challenges in tokenizer construction and annotation quality remain, the MDTID framework is designed to be backward compatible, the Concept Base ID alone reproduces current behavior, while the additional dimensions provide progressive enhancement for systems that choose to leverage them.

12. References

- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL 2016.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.
- Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. EMNLP 2018.
- Xue, L., et al. (2022). ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models. TACL 2022.
- Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.